

Chương 2

Thuật toán & Tư duy lập trình

HUY INIT

Posts and Telecommunications Institute of Technology (PTIT)
Department of Computer Science



**Nguyen Quang
Huy, M.Sc.**

Academic Background



**Lecturer, Faculty of Information Technology,
PTIT**

Sep. 2026 – Present



M.Sc. in Computer Science, PTIT

May 2026



M.Sc. in Information Science, JAIST

Sep. 2026

Research Interests



Computer Vision



Educational Technology (EdTech)



Recommender Systems



Explainable Artificial Intelligence (XAI)



Continual Learning

Contact Information

- **Github:** <https://github.com/huyinit>
- **Facebook:** <https://www.facebook.com/huyinit13>
- **Instagram:** <https://www.instagram.com/huyinit/>
- **Youtube Bài Giảng:** <https://www.youtube.com/@huyinit>
- **Youtube Vlog:** <https://www.youtube.com/@huyinitvlog>

Seeking Research Collaboration

I welcome collaboration with researchers, graduate and undergraduate students, and industry professionals working in AI, RS, XAI, Continual Learning, and related fields. Whether you are interested in joint publications, undergraduate or graduate thesis projects, research internships, research discussions, or interdisciplinary collaborations, I would be happy to connect and explore new ideas together. Feel free to contact me at huyng@ptit.edu.vn.

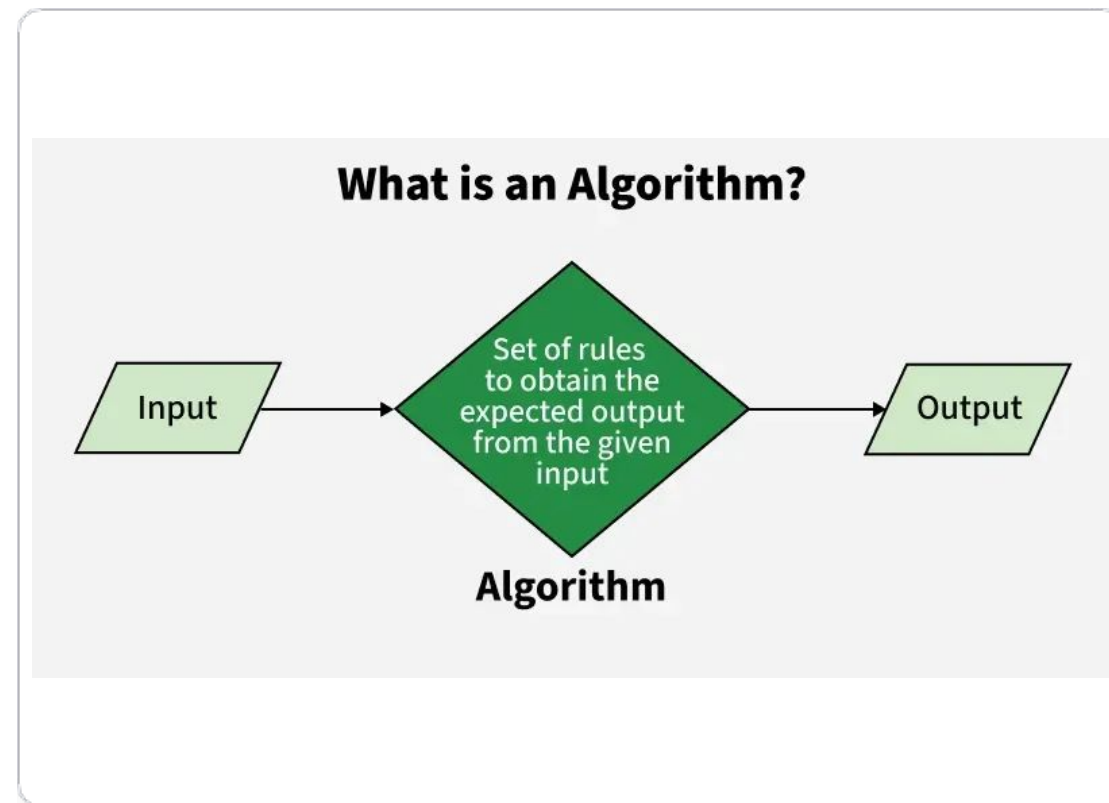
2.1. Khái niệm thuật toán và chương trình

Khái niệm

Khái niệm khoa học: Thuật toán

(Algorithm) là một dãy hữu hạn các bước được xác định rõ ràng nhằm giải quyết một bài toán hoặc thực hiện một nhiệm vụ.

Bản chất: Là cách chúng ta thiết lập logic để hướng dẫn máy tính, bước này qua bước khác, hoàn thành một tác vụ cụ thể.



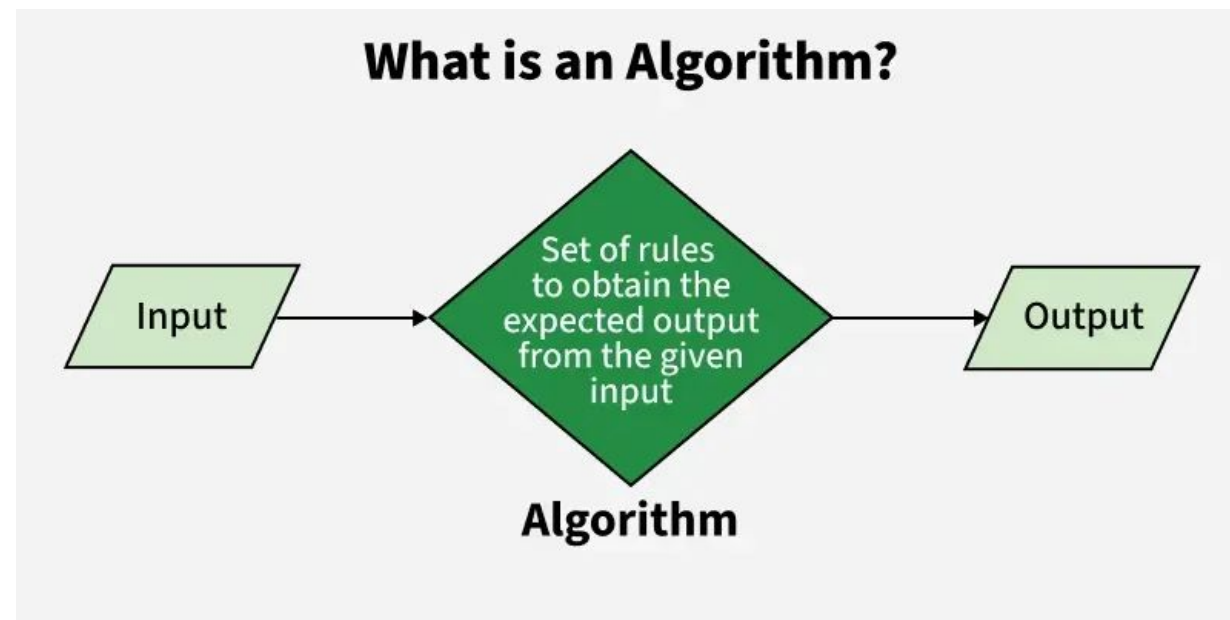
2.1. Khái niệm thuật toán và chương trình

Khái niệm

Thuật toán **F** giải bài toán **P** là dãy các thao tác sơ cấp $F_1, F_2..F_n$ trên tập dữ kiện đầu vào (Input) để đưa ra kết quả (Output).

$F_1F_2..F_n$ (Input) $\rightarrow$ Output.

- **F = $F_1F_2..F_n$** : Thuật toán giải bài toán P (gồm các phép tính số học/logic).
- **Input**: Tập dữ kiện đầu vào.
- **Output**: Kết quả nhận được sau khi thực hiện thuật toán.



2.1. Khái niệm thuật toán và chương trình

Các đặc trưng của thuật toán

Tính đơn định Ở mỗi bước, thao tác sơ cấp phải rõ ràng, không đa nghĩa. Với một tập dữ liệu vào, thuật toán chỉ cho duy nhất một kết quả ra.

Tính dừng Thuật toán không được rơi vào quá trình vô hạn. Phải dừng lại và cho kết quả sau một số hữu hạn các bước tính toán.

Tính đúng Khi thực hiện đúng quy trình, kết quả nhận được phải chính xác với mọi bộ dữ liệu đầu vào, được kiểm chứng bằng yêu cầu bài toán.

Tính phổ dụng Dễ sửa đổi để thích ứng với bất kỳ bài toán nào trong lớp các bài toán cùng loại và xử lý được trên nhiều tập dữ liệu khác nhau.

Tính khả thi Thuật toán phải dễ hiểu, dễ cài đặt và có thể thực thi được trên máy tính thực tế với một giới hạn thời gian cho phép.

2.1. Khái niệm thuật toán và chương trình

Thuật toán (Algorithm)

Là **ý tưởng và thiết kế**, độc lập hoàn toàn với bất kỳ ngôn ngữ lập trình nào. Nó là cách giải quyết bài toán cốt lõi.

Chương trình (Program)

Là thuật toán đã được **cài đặt**
(implement) bằng một ngôn ngữ lập
trình cụ thể (C++, Java, Python...) để
máy tính thực thi.

2.2. Giả mã và Lưu đồ

Phương pháp biểu diễn thuật toán

Ngôn ngữ tự nhiên

Sử dụng phương tiện giao tiếp giữa con người với nhau để biểu diễn các bước của thuật toán một cách tường minh.

Ngôn ngữ hình thức

Phương tiện giao tiếp giữa người
& máy tính (Sơ đồ khối, mã giả).
Gắn gũi với cả tư duy con người
và hệ thống logic của máy.

Ngôn ngữ máy tính

Sử dụng trực tiếp các ngôn ngữ lập trình (C, Python, Java...) để giao tiếp và mô tả chính xác thuật toán cho máy tính hiểu.

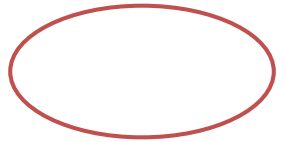
Ghi chú. Trong các phương pháp biểu diễn thuật toán, phương pháp biểu diễn bằng ngôn ngữ hình thức được sử dụng rộng rãi vì nó gần với ngôn ngữ tự nhiên và gần với ngôn ngữ hình thức

2.2. Giải mã và Lưu đồ

- **Ngôn ngữ tự nhiên:** Dễ hiểu nhưng thường dài dòng và có thể gây hiểu nhầm về logic lập trình.
- **Giải mã (Pseudocode):** Ngôn ngữ lai giữa ngôn ngữ tự nhiên và cấu trúc lập trình. Giúp tập trung vào logic cốt lõi mà không bị ràng buộc bởi cú pháp code khắt khe.
- **Lưu đồ (Flowchart):** Biểu diễn luồng thực thi bằng ký hiệu đồ họa (Hình chữ nhật: Xử lý, Hình thoi: Điều kiện, Hình oval: Bắt đầu/Kết thúc).

2.2. Giải mã và Lưu đồ

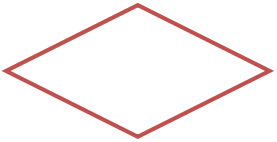
Sử dụng lưu đồ khối



Khối giới hạn
Chỉ thị bắt đầu và kết thúc



Khối vào ra
Nhập xuất dữ liệu



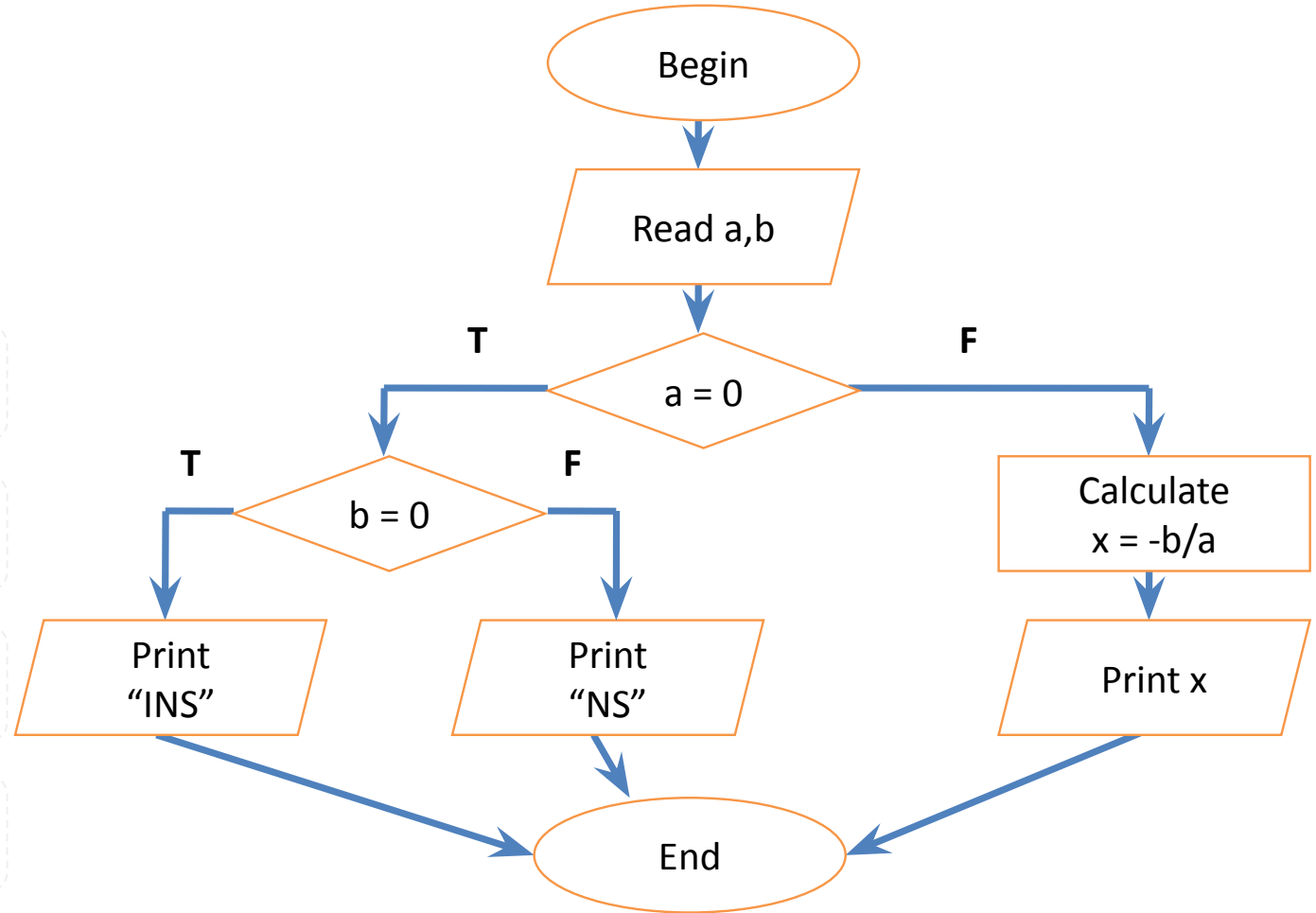
Khối lựa chọn
Tùy điều kiện sẽ rẽ nhánh



Khối thao tác
Ghi thao tác cần thực hiện



Đường đi
Chỉ hướng thao tác tiếp theo



2.2. Giả mã và Lưu đồ

Mã giả (Pseudocode) là một phương pháp dựa trên văn bản để ghi lại thuật toán, kết hợp ngôn ngữ tự nhiên với ngôn ngữ lập trình.

- Sử dụng kết hợp từ ngữ tự nhiên và từ khóa lệnh của ngôn ngữ lập trình để mô tả các hành động mà thuật toán sẽ thực hiện.
- Tổ chức như một dàn ý, sử dụng các mức thụt lề khác nhau để chỉ ra luồng hành động của chương trình.
- Không có từ vựng chuẩn hóa; các từ được chọn phải mô tả rõ ràng các hành động mà thuật toán sẽ thực hiện

```
If a = 0 Then
Begin
    If b = 0 Then
        Print "Phương trình có vô số nghiệm"
    Else
        Print "Phương trình vô nghiệm"
Else
    Print "Phương trình có nghiệm  $x = -b/a$ "
End
```

2.2. 3 Cấu trúc điều khiển



1. Tuần tự

Mặc định máy tính thực hiện các lệnh lần lượt từ trên xuống dưới.
(VD: Nhập a, nhập b, tính $c = a + b$, rồi xuất c).



2. Rẽ nhánh

Chọn 1 trong các hướng đi dựa trên Điều kiện (IF-ELSE). (VD: Nếu n chia hết cho 2 thì n là số chẵn, ngược lại là số lẻ).



3. Lặp

Một nhóm thao tác được yêu cầu thực hiện lặp đi lặp lại nhiều lần (FOR/WHILE). (VD: Duyệt qua danh sách sinh viên).

2.3. Độ phức tạp tính toán (Big-O)

Độ phức tạp tính toán cho biết **thời gian** hoặc **bộ nhớ** mà chương trình cần khi kích thước dữ liệu đầu vào (n) tăng lên.

Thường dùng ký hiệu **Big-O** để biểu diễn:

- **$O(1)$** : Hằng số (Nhanh nhất)
- **$O(\log n)$** : Logarit
- **$O(n)$** : Tuyến tính
- **$O(n \log n)$** : Gần tuyến tính
- **$O(n^2)$** : Bình phương
- **$O(2^n)$** : Hàm mũ (Chậm nhất)

2.3. Độ phức tạp tính toán (Big-O)

$O(1)$ – Hằng số

Truy cập trực tiếp phần tử trong List theo chỉ số. Dù danh sách có 10 hay 1 triệu phần tử, thời gian truy cập vẫn không đổi.

```
a = [10, 20, 30, 40, 50]
print(a[2]) # Truy cập a[2]
```

→ $O(1)$

$O(n)$ – Tuyến tính

Duyệt qua các phần tử của List. Nếu có n phần tử thì vòng lặp sẽ phải chạy đúng n lần.

```
a = [10, 20, 30, 40, 50]
for x in a:
    print(x)
```

→ $O(n)$

2.3. Độ phức tạp tính toán (Big-O)

$O(n^2)$ – Bình phương

Hai vòng lặp lồng nhau. Vòng ngoài chạy n lần, mỗi lần vòng ngoài thì vòng trong cũng chạy n lần ($n \times n = n^2$).

```
a = [1, 2, 3, 4, 5]
for x in a:
    for y in a:
        print(x, y)
```

→ $O(n^2)$

$O(\log n)$ – Logarit

Tìm kiếm nhị phân (Binary Search). Ở mỗi bước, thuật toán loại bỏ đi một nửa số lượng dữ liệu (VD: 1.000.000 → 500.000 → 250.000...)

```
while left <= right:
    mid = (left + right) // 2
    if a[mid] == target:
        return mid
    # ... (Cắt đôi khoảng tìm kiếm)
```

→ $O(\log n)$

2.3. Độ phức tạp tính toán (Big-O)

Các thao tác có sẵn của Python **không** phải lúc nào cũng là $O(1)$.



List (Danh sách)

Dùng toán tử in trên List mất thời gian $O(n)$ (trường hợp xấu nhất). Python phải duyệt qua từng phần tử từ đầu đến cuối để kiểm tra xem giá trị có tồn tại hay không.

`10 in [10, 20, 30] →  $O(n)$`



Dictionary (Từ điển)

Dictionary sử dụng Hash Table (Bảng băm). Việc truy cập bằng khóa (Key) hoặc kiểm tra khóa tồn tại diễn ra cực kỳ nhanh chóng với thời gian trung bình là $O(1)$.

`"name" in dict →  $O(1)$`



Set (Tập hợp)

Tương tự như Dictionary, Set cũng dùng Hash Table. Kiểm tra một phần tử có trong Set hay không chỉ mất trung bình $O(1)$. Rất hữu ích để tối ưu thuật toán!

`30 in set →  $O(1)$`

2.3. Độ phức tạp tính toán (Big-O)

Cách 1: Dùng vòng lặp (Chậm)

Sử dụng in trên list b bên trong một vòng lặp duyệt qua list a.

```
for x in a:
    if x in b: # O(n)
        print(x)
```

- Duyệt a: $O(n)$
- Kiểm tra $x \in b$: $O(n)$

Tổng thể: $O(n^2)$

Cách 2: Dùng Set (Nhanh)

Chuyển list b thành Set trước. Việc tra cứu trong Set chỉ mất $O(1)$.

```
b_set = set(b) # O(n)

for x in a:
    if x in b_set: # O(1)
        print(x)
```

- Tạo $\text{set}(b)$: $O(n)$
- Duyệt a & Tra cứu: $n \times O(1) = O(n)$

Tổng thể: $O(n)$

2.4. Quy trình phân tích bài toán

Các bước tiến hành giải bài toán



2.5. Kiểm thử, gỡ lỗi và kiểm thử tự động trong học lập trình

Mối quan hệ giữa Kiểm thử và Gỡ lỗi

Kiểm thử (Testing)

Quá trình chạy phần mềm với mục đích **tìm ra lỗi (bug)**. Nó trả lời câu hỏi: *"Chương trình có hoạt động đúng như mong đợi không?"*

-  Có thể thực hiện thủ công hoặc tự động.
-  Dựa trên các Test Case đã định sẵn.
-  Chỉ ra sự hiện diện của lỗi, không chỉ ra vị trí.

Gỡ lỗi (Debugging)

Quá trình **cô lập và sửa chữa** nguyên nhân gây ra lỗi. Nó trả lời câu hỏi: *"Tại sao chương trình lại sai và sửa thế nào?"*

-  Thường do chính Lập trình viên thực hiện.
-  Phân tích từng dòng code, xem xét bộ nhớ.
-  Kết thúc khi lỗi được khắc phục hoàn toàn.

2.5. Kiểm thử, gỡ lỗi và kiểm thử tự động trong học lập trình

Phân loại lỗi lập trình với Python



Lỗi Cú pháp (Syntax Error)

Code sai ngữ pháp Python.
Chương trình **không thể chạy** được.

```
if x == 10 # Thiếu ':'  
    print("Đúng")
```

```
SyntaxError: expected ':'
```



Lỗi Thực thi (Runtime Error)

Cú pháp đúng, nhưng chạy bị **sập** (crash) do dữ liệu không hợp lệ.

```
a = [1, 2, 3]  
print(a[5]) # Quá giới hạn
```

```
IndexError: out of range
```



Lỗi Logic (Logic Error)

Code chạy bình thường, không báo lỗi, nhưng **kết quả lại sai**.

```
def tong(a, b):  
    return a * b # Sai phép tính  
  
print(tong(2, 3)) # Ra 6, cần 5
```

2.5. Kiểm thử, gỡ lỗi và kiểm thử tự động trong học lập trình

Kỹ thuật Gỡ lỗi (Debugging Skills)

-  **Sử dụng lệnh In (Print Debugging):** Cách cơ bản nhất. Chèn các lệnh `print()` vào giữa các bước xử lý để xem giá trị của biến tại thời điểm đó là gì, giúp khoanh vùng dòng code gây lỗi.
-  **Sử dụng công cụ Debugger của IDE:** Các trình soạn thảo (VS Code, PyCharm) có công cụ Debugger cực mạnh. Cho phép đặt **Breakpoint** (điểm dừng) và chạy code từng dòng để theo dõi sự thay đổi của bộ nhớ.
- Phương pháp "Vịt cao su" (Rubber Duck Debugging):** Giải thích từng dòng code của bạn cho một con vịt cao su (hoặc người khác). Việc đọc to logic giúp não bộ tự nhận ra những điểm vô lý mà mắt thường bỏ qua.

2.5. Kiểm thử, gỡ lỗi và kiểm thử tự động trong học lập trình

Phân loại Test Cases

Tại sao cần Test Case?

Một lập trình viên chuyên nghiệp không chỉ hỏi "code có chạy được không?", mà phải hỏi "**code có đúng trong mọi trường hợp không?**".

Một thuật toán sai logic có thể vẫn chạy đúng với vài bộ dữ liệu ngẫu nhiên.

- **Trường hợp thông thường:**
Giá trị trung bình (VD: $n = 10$).
- **Trường hợp biên:**
Giá trị cực tiểu, cực đại (VD: $n = 0$, $n = 10^9$).
- **Trường hợp ngoại lệ:**
Số âm, mảng rỗng. Đây là nơi code dễ "chết" (crash) nhất.

2.5. Kiểm thử, gỡ lỗi và kiểm thử tự động trong học lập trình

Kiểm thử tự động cơ bản với lệnh assert

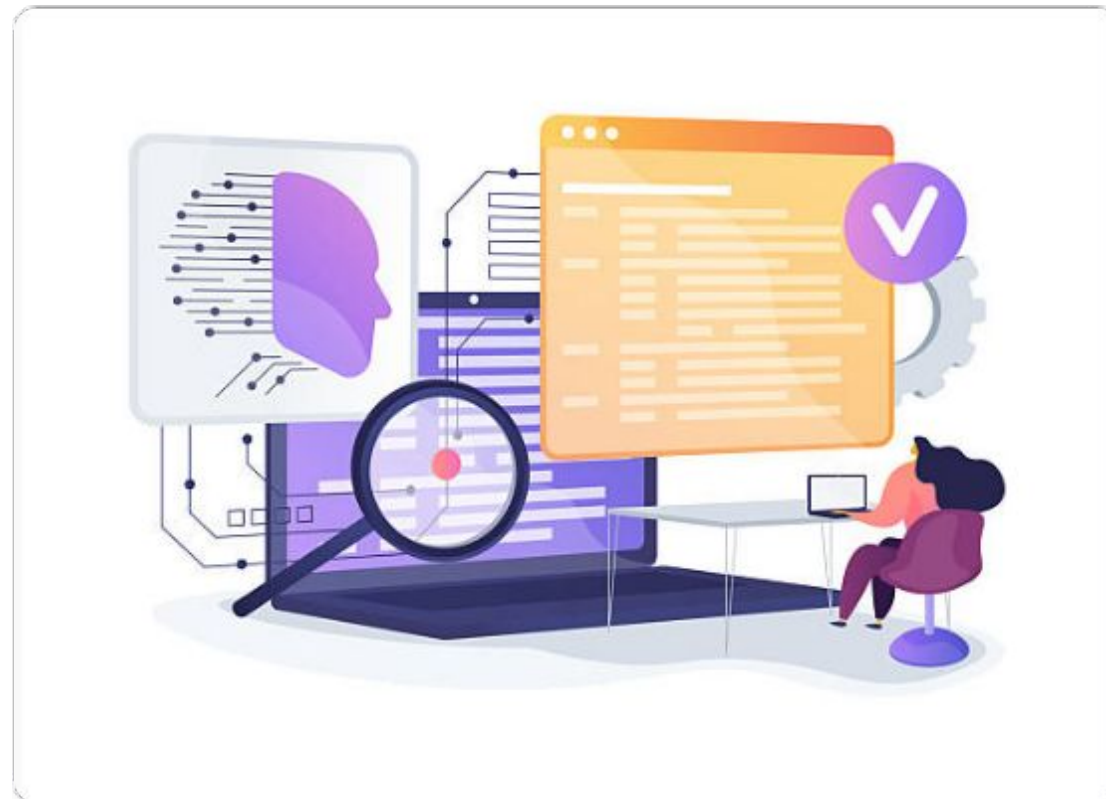
Nhập dữ liệu bằng tay để test rất mất thời gian. Chúng ta nên viết code để máy tính tự test chương trình của mình.

Cú pháp lệnh `assert`:

```
assert <điều_kiện> ,
```

Nếu điều kiện đúng (True), chương trình đi tiếp. Nếu sai (False), chương trình dừng ngay lập tức.

```
def is_even(n):  
    return n % 2 == 0  
  
# Tự động kiểm tra  
assert is_even(4) == True, "Lỗi chẵn"  
assert is_even(7) == False, "Lỗi lẻ"
```



2.5. Kiểm thử, gỡ lỗi và kiểm thử tự động trong học lập trình

Kiểm thử chuyên nghiệp với unittest

Thư viện chuẩn của Python

Trong thực tế doanh nghiệp, Lập trình viên sử dụng các framework có sẵn như unittest hoặc pytest để quản lý hàng nghìn test cases.

Cơ chế hoạt động:

- ⚙️ Kế thừa class `TestCase`.
- ⚡ Viết các hàm bắt đầu bằng `test\_`.
- 📄 Tự động sinh báo cáo tổng hợp.

```
import unittest

class TestMath(unittest.TestCase):
    def test_add(self):
        self.assertEqual(2 + 3, 5)

    def test_sub(self):
        self.assertEqual(10 - 5, 5)

if __name__ == '__main__':
    unittest.main()
```

Tư tưởng cốt lõi

Lập trình giỏi không bắt đầu từ việc gõ phím thật nhanh.

Nó bắt đầu từ việc **hiểu sâu bài toán**, xây dựng **logic thuật toán** đúng đắn trên giấy, và biết cách **kiểm chứng, gỡ lỗi** một cách khoa học.